

Code Explanation

Code Explanation: AWS Glue PySpark ETL Job
This code implements a complete ETL (Extract, Transform, Load) pipeline for processing customer and order data using AWS Glue and PySpark. Here's a clear breakdown:---

****High-Level Overview****

The job performs these operations:
1. Reads customer and order CSV files from S3
2. Cleans the data (removes nulls, duplicates)
3. Writes cleaned data to Parquet format
4. Registers tables in AWS Glue Data Catalog
5. Calculates customer spending aggregates
6. Writes and catalogs the aggregated results---

****Component Breakdown****

****1. SparkContextManager****

****Purpose:**** Initializes the Spark and Glue runtime environments
****Key Method:**** `initialize_spark_contexts()`
- Detects if running in AWS Glue or locally
- Creates SparkContext and SparkSession
- Initializes GlueContext for AWS Glue operations
- Attempts to initialize a Glue Job (fails gracefully if running locally)
- Returns: `(spark, glueContext, job)` tuple---

****2. ConfigManager****

****Purpose:**** Loads job configuration from YAML file
****Key Methods:**** `load_config()`: Reads and parses YAML configuration file
`get_job_parameters()`: Wrapper that returns configuration dictionary
****Configuration includes:****
- S3 paths for source/target data
- File formats (CSV, Parquet)
- Table names for Glue Catalog
- Data cleaning flags
- Expected column schemas---

****3. S3DataReader****

****Purpose:**** Safely reads data from S3 with validation
****Key Method:**** `read_data_safe()`
****Steps:****
1. Validates the S3 path is not empty
2. Reads data based on format (CSV or Parquet)
3. Normalizes all column names to lowercase
4. Validates schema if expected columns are provided
5. Logs row count (warns if empty)
6. Returns DataFrame
****Error Handling:****
- Catches path not found errors
- Catches schema validation failures
- Catches parsing errors---

****4. S3DataWriter****

****Purpose:**** Safely writes DataFrames to S3
****Key Method:**** `write_data_safe()`
****Steps:****
1. Logs the write operation details
2. Counts rows being written
3. Writes data in specified format (Parquet or CSV)
4. Supports write modes: overwrite, append, etc.
5. Logs success with row count---

****5. DataCleaner****

****Purpose:**** Performs data quality operations
****Key Methods:****
`remove_nulls_and_null_strings()`: Removes rows with NULL values in any column. Also removes rows with string literal "Null". Builds a filter condition checking all columns. Logs how many rows were removed
`remove_duplicates()`: Uses Spark's `dropDuplicates()` to remove duplicate rows. Keeps first occurrence. Logs how many duplicates were removed
`clean_dataframe()`: Orchestrates both cleaning operations. Can be

disabled via configuration flag---

****6. GlueCatalogManager****

****Purpose:**** Manages AWS Glue Data Catalog table registration****Key Method:**** ``register_table()``****Steps:****1. Creates the database if it doesn't exist2. Drops the table if it already exists3. Writes DataFrame as Parquet to S34. Registers it as an external table in Glue Catalog5. Makes data queryable via Athena/Glue---

****7. DataAggregator****

****Purpose:**** Performs business logic transformations****Key Method:**** ``calculate_customer_aggregate_spend()``****Steps:****1. Joins customer and order DataFrames on ``custid``2. Calculates ``totalamount = priceperunit × qty``3. Groups by: ``custid`, `name`, `emailid`, `region`, `date``4. Sums ``totalamount`` to get ``totalspend`` per group5. Returns aggregated DataFrame---

****Main ETL Pipeline Execution****

The ``main()`` function orchestrates the entire workflow:

****Step 1: Initialize Environment****

- Creates Spark/Glue contexts
- Loads configuration from YAML
- Initializes reader, writer, and catalog manager

****Step 2: Ingest Customer Data****

- Reads customer CSV from S3
- Validates expected columns
- Returns raw customer DataFrame

****Step 3: Ingest Order Data****

- Reads order CSV from S3
- Validates expected columns
- Returns raw order DataFrame

****Step 4: Clean Customer Data****

- Removes nulls and "Null" strings
- Removes duplicates
- Returns cleaned customer DataFrame

****Step 5: Clean Order Data****

- Applies same cleaning operations
- Returns cleaned order DataFrame

****Step 6: Write & Catalog Customer Data****

- Writes cleaned customer data to Parquet
- Registers as table in Glue Catalog
- Now queryable via Athena

****Step 7: Write & Catalog Order Data****

- Writes cleaned order data to Parquet
- Registers as table in Glue Catalog

****Step 8: Calculate Aggregates****

- Joins customer and order data- Calculates total spending per customer per date- Returns aggregated DataFrame

****Step 9: Write & Catalog Aggregates****

- Writes aggregate results to Parquet- Registers as table in Glue Catalog- Commits the Glue job---

****Key Design Patterns****

1. ****Separation of Concerns:**** Each class handles one responsibility2. ****Safe Operations:**** All I/O operations include error handling and logging3. ****Configuration-Driven:**** All parameters externalized to YAML4. ****Validation:**** Schema validation ensures data quality5. ****Logging:**** Comprehensive logging at each step for debugging6. ****Flexibility:**** Can run in AWS Glue or locally for testing---

****Error Handling Strategy****

- ****Path errors:**** Caught and logged with clear messages- ****Schema errors:**** Validates expected columns exist- ****Parsing errors:**** Catches format-specific failures- ****Job failures:**** Logs error and re-raises for Glue to handle- ****Graceful degradation:**** Job initialization fails gracefully for local runs---

****Output Artifacts****

After successful execution:1. ****Cleaned customer data**** in Parquet format2. ****Cleaned order data**** in Parquet format3. ****Aggregated spending data**** in Parquet format4. ****Three tables**** registered in Glue Data Catalog5. ****All data queryable**** via Amazon Athena